

Infrastructure Monitoring using Prometheus and Grafana on AWS for System and Application.

Follow below steps to implement the micro service observability tech stack.

Prerequisites

- Ubuntu (choose accordingly)
- Root user account with **sudo** privilege.
- Prometheus system user and group.
- Sufficient storage on your system and good internet connectivity.
- Ports Required- **9090 (Prometheus)**, **3000 (Grafana)**, **9100 (Node Exporter)**, **8001 (microservice)**

Set up a Sample Microservices Application(Deploy a demo app with multiple services (e.g., Banking App with balance-service, payment-service)

Enable Logging (Structured + Centralized)(NodeExporter + Prom + Grafana.)

Collect Metrics (Install Prometheus to scrape service metrics (expose /metrics endpoint with libraries like prometheus-client).)

.....

Lets start with the exercise.

Set Up EC2 Instance in AWS Console (Navigate to ec2)

Click > Launch Instance > Provide Name > Select Ubuntu Image > Instance Type (t2.micro) > Create Key Pair > Create Security Group (Allow SSH Traffic) > Launch Instance **Ensure your EC2 instance is up and running:**
Launch an EC2 Instance (Ubuntu).Open Inbound Ports:

- SSH (22)
- Prometheus (9090)

- Grafana (3000)
- Payment Service (8001)
- Balance Service (8001)

SSH to system:

```
chmod 400 SRE_Demo.pem  
ssh -i "sre_demo1.pem" ubuntu@ec2-<public-ip>.us-east-2.compute.amazonaws.com
```

We will update the system repository index by using the following command.

```
sudo apt update -y
```

Step #1: Creating Prometheus System Users and Directory

Create a system user for Prometheus using below command:

```
sudo useradd --no-create-home --shell /bin/false prometheus
```

Create the directories in which we will be storing our configuration files and libraries:

```
sudo mkdir /etc/prometheus  
sudo mkdir /var/lib/prometheus
```

Set the ownership of the /var/lib/prometheus directory with below command:

```
sudo chown prometheus:prometheus /var/lib/prometheus
```

Step #2: Download Prometheus Binary File

Using below command we can download Prometheus, here we are downloading Prometheus 2.46 version, you use above link to download specific version.

Lets create a folder say /tmp :

```
cd /tmp/
```

Download the Prometheus setup using wget

```
wget  
https://github.com/prometheus/prometheus/releases/download/v2.46.0/prometheus-2.46.0.linux-amd64.tar.gz
```

Extract the files using tar :

```
tar -xvf prometheus-2.46.0.linux-amd64.tar.gz
```

Move the configuration file and set the owner to the prometheus user:

```
cd prometheus-2.46.0.linux-amd64  
sudo mv console* /etc/prometheus  
sudo mv prometheus.yml /etc/prometheus  
sudo chown -R prometheus:prometheus /etc/prometheus
```

Now, Move the binaries and set the owner:

```
sudo mv prometheus /usr/local/bin/  
sudo chown prometheus:prometheus /usr/local/bin/prometheus
```

Step #3:Prometheus configuration file

We have already copied **/opt/prometheus-2.26.0.linux-amd64/prometheus.yml** file **/etc/prometheus** directory, verify if it present and should look like below and modify it as per your requirement.

```
sudo vi /etc/prometheus/prometheus.yml
```

(Don't change any config yet)

Step #4: Creating Prometheus Systemd file

Create the service file using below command:

```
sudo vi /etc/systemd/system/prometheus.service
```

[Unit]

Description=Prometheus

Wants=network-online.target

After=network-online.target

[Service]

User=prometheus

Group=prometheus

Type=simple

ExecStart=/usr/local/bin/prometheus \

--config.file /etc/prometheus/prometheus.yml \

--storage.tsdb.path /var/lib/prometheus/ \

--web.console.templates=/etc/prometheus/consoles \

--web.console.libraries=/etc/prometheus/console_libraries

[Install]

WantedBy=multi-user.target

Now, Reload systemd:

```
sudo systemctl daemon-reload
```

Start, Enable, Status Prometheus service:

```
sudo systemctl start prometheus
```

```
sudo systemctl enable prometheus
```

```
sudo systemctl status prometheus
```

```

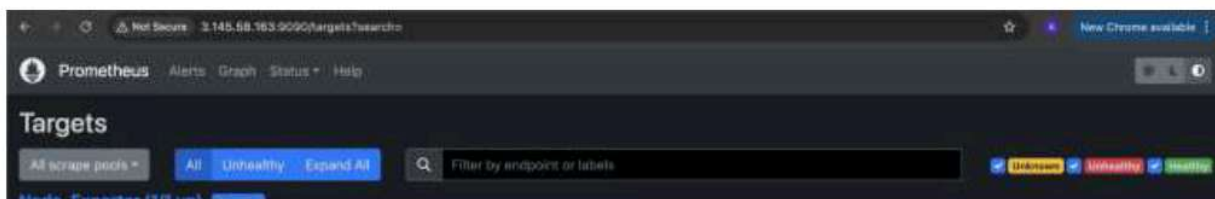
prometheus-2.46.0-linux-amd64/console/prometheus-overview.html
prometheus-2.46.0-linux-amd64/console/node.html
prometheus-2.46.0-linux-amd64/console/index.html.example
prometheus-2.46.0-linux-amd64/console/node-overview.html
ubuntu@ip-172-31-0-24:~$ cd prometheus-2.46.0-linux-amd64
sudo mv console /etc/prometheus
sudo mv prometheus.yml /etc/prometheus
sudo chown -R prometheus:prometheus /etc/prometheus
ubuntu@ip-172-31-0-24:~$ cd /usr/local/bin/
sudo mv prometheus /usr/local/bin/prometheus
ubuntu@ip-172-31-0-24:~$ cd /etc/prometheus/
sudo vi /etc/prometheus/prometheus.yml
ubuntu@ip-172-31-0-24:~$ cd /etc/systemd/system/
sudo vi /etc/systemd/system/prometheus.service
ubuntu@ip-172-31-0-24:~$ sudo systemctl daemon-reload
ubuntu@ip-172-31-0-24:~$ sudo systemctl start prometheus
sudo systemctl enable prometheus
sudo systemctl status prometheus
Created symlink /etc/systemd/system/multi-user.target.wants/prometheus.service → /etc/systemd/system/prometheus.service.
● prometheus.service - Prometheus
   Loaded: loaded /etc/systemd/system/prometheus.service; enabled; preset: enabled)
   Active: active (running) since Wed 2020-01-29 09:51:05 UTC; 35ms ago
     Main PID: 1656 (prometheus)
       Tasks: 6 (limit: 1150)
      Memory: 15.5M (peak: 15.8M)
         CPU: 5ms
        CGroup: /system.slice/prometheus.service
               └─1656 /usr/local/bin/prometheus --config.file /etc/prometheus/prometheus.yml --storage.tsdb.path /var/lib/prometheus/ --web.console.templates=/etc/prom

```

Step #5: Accessing Prometheus in Browser

Now as Prometheus installation and configuration is set up and it is ready to use we can access its services via web interface. Also check whether port 9090 is UP in firewall.

Now Prometheus service is ready to run and we can access it from any web browser.



<http://server-IP-or-Hostname:9090>

Step #6: Download Node Exporter

Go to the official release page of Prometheus Node Exporter and copy the link of the latest version of the Node Exporter package according to your OS type.


```
cd /tmp
```

Now lets run the copied URL with **wget** command

```
wget  
https://github.com/prometheus/node\_exporter/releases/download/v1.6.1/node\_exporter-1.6.1.linux-amd64.tar.gz
```

Unzip the downloaded the file using below command

```
sudo tar xvfz node_exporter-*.linux-amd64.tar.gz
```

Move the binary file of node exporter to /usr/local/bin location

```
sudo mv node_exporter-*.linux-amd64/node_exporter /usr/local/bin/
```

Create a node_exporter user to run the node exporter service

```
sudo useradd -rs /bin/false node_exporter
```

Step #7:Creating Node Exporter Systemd service

Create a node_exporter service file in the /etc/systemd/system directory

```
sudo vi /etc/systemd/system/node_exporter.service
```

[Unit]

Description=Node Exporter

After=network.target

[Service]

User=node_exporter

Group=node_exporter

Type=simple

ExecStart=/usr/local/bin/node_exporter

[Install]

WantedBy=multi-user.target

Now lets start and enable the node_exporter service using below commands

```
sudo systemctl daemon-reload
sudo systemctl start node_exporter
sudo systemctl enable node_exporter
```

```
root@kali:~# sudo systemctl daemon-reload
root@kali:~# sudo systemctl start node_exporter
root@kali:~# sudo systemctl enable node_exporter
root@kali:~# sudo systemctl status node_exporter
Created symlink /etc/systemd/system/multi-user.target.wants/node_exporter.service → /etc/systemd/system/node_exporter.service.
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-01-29 09:46:42 UTC; 30ms ago
     Main PID: 1792 (node_exporter)
        Task: 3 (limit: 1136)
      Memory: 2.1M (peak: 3.3M)
         CPU: 4ms
    CGroup: /system.slice/node_exporter.service
            └─1792 /usr/local/bin/node_exporter

Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectorthermal_zone
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectortime
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectortime
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectorvmstat
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectorvmstat
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectorwifi
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=node_exporter.go:117 level=info collectorwifi
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=utils_config.go:276 level=info http.ListenAndServe address=:9100
Jan 29 09:46:42 ip-172-31-6-24 node_exporter[1792]: ts=2025-01-29T09:46:42.044Z caller=utils_config.go:177 level=info http.ListenAndServe address=:9100
```

```
sudo systemctl status node_exporter
```

Step #8: Configure the Node Exporter as a Prometheus target

Now to scrape the node_exporter lets instruct the Prometheus by making a minor change in prometheus.yml file

So go to etc/prometheus and open prometheus.yml

```
sudo vi /etc/prometheus/prometheus.yml
```

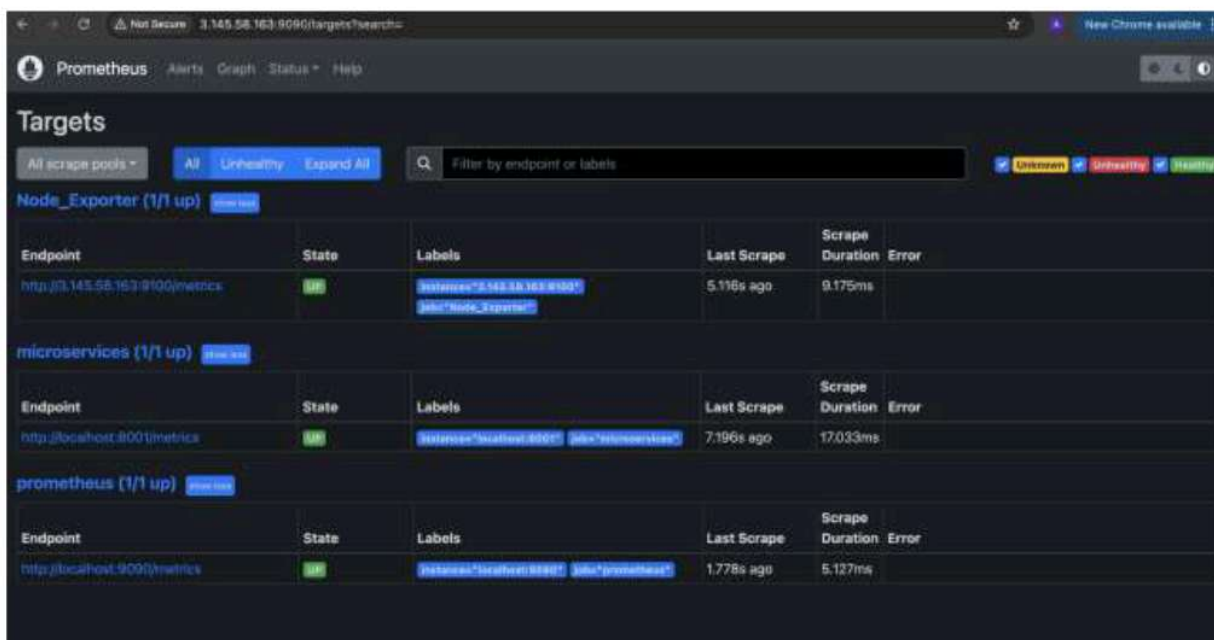
```
- job_name: 'Node_Exporter'
  scrape_interval: 5s
  static_configs:
    - targets: ['<Server_IP_of_Node_Exporter_Machine>:9100']
```

Now restart the Prometheus Service

```
sudo systemctl restart prometheus
```

Hit the URL in your web browser to check whether our target is successfully scraped by Prometheus or not

```
http://server-IP:9100/metrics
```



The screenshot shows the Prometheus web interface with the 'Targets' tab selected. The browser address bar shows the URL `http://3.143.58.163:9090/targets?search=`. The interface includes a search bar and filters for 'All', 'Unhealthy', and 'Expand All'. The targets are organized into three sections: 'Node_Exporter (1/1 up)', 'microservices (1/1 up)', and 'prometheus (1/1 up)'. Each section contains a table with columns for 'Endpoint', 'State', 'Labels', 'Last Scrape', 'Scrape Duration', and 'Error'.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
Node_Exporter (1/1 up)					
<code>http://3.143.58.163:9100/metrics</code>	UP	<code>instance="3.143.58.163:9100"</code> <code>job="Node_Exporter"</code>	5.116s ago	9.175ms	
microservices (1/1 up)					
<code>http://localhost:8001/metrics</code>	UP	<code>instance="localhost:8001"</code> <code>job="microservices"</code>	7.196s ago	17.033ms	
prometheus (1/1 up)					
<code>http://localhost:9090/metrics</code>	UP	<code>instance="localhost:9090"</code> <code>job="prometheus"</code>	1.778s ago	5.127ms	


```
← → ↻ ⚠ Not Secure 3.145.58.163:9100/metrics

# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 2.537e-05
go_gc_duration_seconds{quantile="0.25"} 2.8019e-05
go_gc_duration_seconds{quantile="0.5"} 2.9381e-05
go_gc_duration_seconds{quantile="0.75"} 4.3041e-05
go_gc_duration_seconds{quantile="1"} 0.000527984
go_gc_duration_seconds_sum 0.034544582
go_gc_duration_seconds_count 802
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.20.6"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated and still in use.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.142768e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated, even if freed.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 1.482175896e+09
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.59768e+06
# HELP go_memstats_frees_total Total number of frees.
# TYPE go_memstats_frees_total counter
go_memstats_frees_total 1.8293229e+07
# HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata.
# TYPE go_memstats_gc_sys_bytes gauge
go_memstats_gc_sys_bytes 8.449608e+06
# HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and still in use.
# TYPE go_memstats_heap_alloc_bytes gauge
go_memstats_heap_alloc_bytes 2.142768e+06
# HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used.
# TYPE go_memstats_heap_idle_bytes gauge
go_memstats_heap_idle_bytes 4.317184e+06
# HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use.
# TYPE go_memstats_heap_inuse_bytes gauge
go_memstats_heap_inuse_bytes 3.678208e+06
# HELP go_memstats_heap_objects Number of allocated objects.
# TYPE go_memstats_heap_objects gauge
go_memstats_heap_objects 12910
# HELP go_memstats_heap_released_bytes Number of heap bytes released to OS.
# TYPE go_memstats_heap_released_bytes gauge
go_memstats_heap_released_bytes 3.293184e+06
# HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system.
```

Step #9: Install Grafana on Ubuntu 22.04 LTS

Now lets Install Grafana for wonderful dashboards and data visualization for monitoring systems, servers, services, etc

Add the Grafana GPG key in Ubuntu using wget

```
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
```

Next, add the Grafana repository to your APT sources:

```
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb
stable main"
```

Refresh your APT cache to update your package lists:

```
sudo apt update
```

You can now proceed with the installation:

```
sudo apt install grafana
```

Once Grafana is installed, use `systemctl` to start the Grafana server:

```
sudo systemctl start grafana-server
```

Next, verify that Grafana is running by checking the service's status:

```
sudo systemctl enable grafana-server
```

```
sudo systemctl status grafana-server
```

[illegible]

To access Grafana Dashboard open your favorite browser, type server IP or Name followed by grafana default port 3000.

http://your_ip:3000

Here you can see Login page of Grafana now you will have to login with below Grafana default UserName and Password.

Username – admin

Password – admin

Now, will create MicroService Node.js App which contains Payment and Balance Service.

Step 1: Install Node.js and Dependencies

```
cd /tmp
```

```
mkdir microservice-monitoring && cd microservice-monitoring
```

```
sudo apt install npm
```

```
npm init -y
```

```
npm install express prom-client cors
```

Step 2: Create the Node.js Application

Create a file named **server.js**:

```
sudo vi server.js
```

```
const express = require("express");  
const client = require("prom-client");  
const cors = require("cors");
```

```
const app = express();  
app.use(cors());
```

```
// Prometheus metrics setup  
const register = new client.Registry();
```

```
client.collectDefaultMetrics({ register }); // Collect default system metrics (optional)
```

```
const httpRequestDurationMicroseconds = new client.Histogram({  
  name: "http_request_duration_seconds",  
  help: "Duration of HTTP requests in seconds",  
  labelNames: ["service", "method"],  
  buckets: [0.1, 0.5, 1, 2, 5]  
});  
register.registerMetric(httpRequestDurationMicroseconds);
```

```
// Payment Service  
app.get("/pay", (req, res) => {  
  const start = Date.now();  
  setTimeout(() => {  
    const duration = (Date.now() - start) / 1000;  
    httpRequestDurationMicroseconds  
      .labels("payment", "GET")  
      .observe(duration);  
    res.send("Payment processed!");  
  }, Math.random() * 1000); // Simulate latency  
});
```

```
// Balance Service  
app.get("/balance", (req, res) => {  
  const start = Date.now();  
  setTimeout(() => {  
    const duration = (Date.now() - start) / 1000;  
    httpRequestDurationMicroseconds  
      .labels("balance", "GET")  
      .observe(duration);  
    res.send("Balance: $1000");  
  }, Math.random() * 500); // Simulate latency  
});
```

```
// Prometheus Metrics Endpoint  
app.get("/metrics", async (req, res) => {  
  res.set("Content-Type", register.contentType);  
  res.end(await register.metrics());  
});
```

```
// Start server on port 8001  
app.listen(8001, () => {  
  console.log("Server running on port 8001");  
});
```

```

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any time series scraped from this config.
  - job_name: 'prometheus'

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ['localhost:9090']

      - job_name: 'node_exporter'
        static_configs:
          - targets: ['<hostname>:9100']

      - job_name: 'microservices'
        static_configs:
          - targets: ['<hostname>:8001']

```

Step 3: Run the Application

node server.js

This command allows your process to continue running after you log out or close the terminal.

nohup node server.js &

Now, again Configure **prometheus.yml** and add :

```
sudo vi /etc/prometheus/prometheus.yml
```

```

scrape_configs:
  - job_name: "microservices"
    static_configs:
      - targets: ["<hostname-service_ip>:8001"]

```

Now, Restart the Prometheus Server

```
sudo systemctl restart prometheus
```

This will:

- Start a **simple backend**
- Expose:
 - `http://<hostname-service_ip>:8001/pay`

- `http://<hostname-service_ip>:8001/balance`
- `http://<hostname-service_ip>/metrics` (for Prometheus)

`curl http://localhost:8001/metrics`



Set Up Grafana to Visualize

Metrics

Now, let's set up Grafana to visualize the metrics.

1. **Log in to Grafana:** Open Grafana in your browser at `http://localhost:3000` (default username is `admin` and password is `admin`).
2. **Add Prometheus as a Data Source:**
 - In the left sidebar, click on the **gear icon** (⚙️) to go to **Configuration**.
 - Click on **Data Sources** and then click **Add data source**.
 - Select **Prometheus** from the list.
 - In the **HTTP** section, set the URL to your Prometheus server, for example:
`http://localhost:9090`

Click **Save & Test** to verify the connection.

Create a Dashboard:

- In the left sidebar, click the **plus icon (+)** and select **Dashboard**.

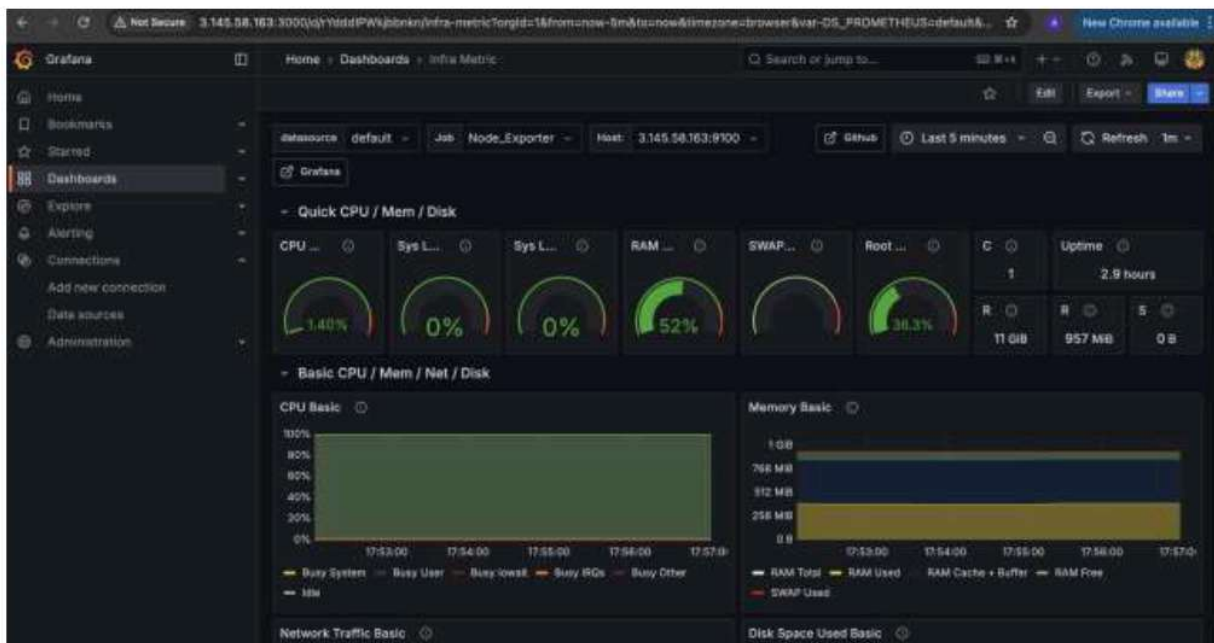
Creating Grafana Dashboard to Monitor Linux Server

Now let's build a dashboard in Grafana so then it will be able to reflect the metrics of the Ubuntu system (CPU, Memory etc)

So we will use 14513 to import from Grafana.com, Let's come to Grafana Home page and you can see a “+” icon. Click on that and select “Import”

Now provide the Grafana.com Dashboard ID which is **14513** and click on **Load**

There you are done with the setup. Now your Dashboard is running up!.



To check for Balance and Payment Service http request and application latency.

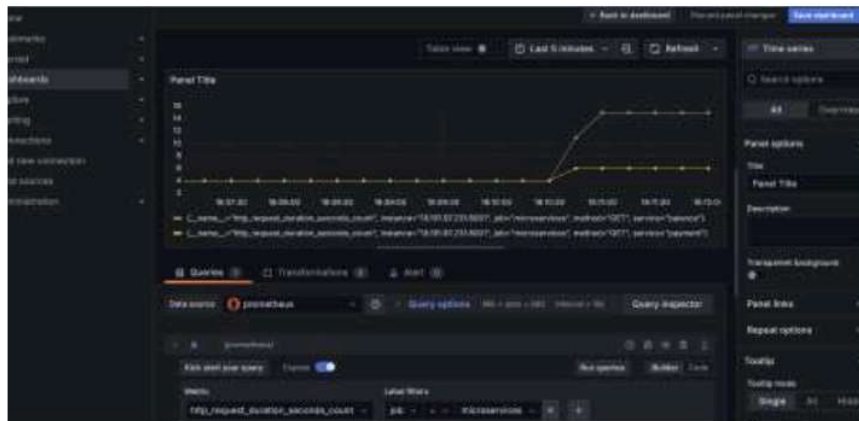
In the left sidebar, click the plus icon (+) and select Dashboard. Click Add new panel.

In query section: add

`http_requests_total_count`

or

`http_request_duration_seconds_count`



Now, You should have successfully implemented Infrastructure Monitoring using Prometheus and Grafana on AWS.

Note:

Basic Troubleshooting command:

```
sudo netstat -tulnp | grep 3001
```

```
ps aux | grep node
```

```
curl -v http://localhost:3001/metrics
```

```
DEBUG=* node server.js
```

```
sudo journalctl -u grafana-server -f
```

Run the following command to check for errors in your Prometheus config:

```
prometheus --config.file=/etc/prometheus/prometheus.yml --log.level=debug
```

If the issue persists, check logs for a more specific error message:

```
journalctl -u prometheus --no-pager | tail -20
```

